Decentralized, Browser-Based Docker IDEs for C++ and x86-64 Assembly Education: An Evaluation on the Laptops Students Actually Own

Sean Cuenco
Department of Computer Science, University of Nevada, Las Vegas
Faculty Research Mentor: James Andro-Vasko, Ph.D.
Department of Computer Science, University of Nevada, Las Vegas

Summer Undergraduate Research Fellowship 2026
Office of Undergraduate Research, UNLV

Code, handouts, test script, raw results:
github.com/seancnc003/UNLVDockerIDEs
hub.docker.com/r/seancnc/unlv-cpp-ide
hub.docker.com/r/seancnc/unlv-x86-ide

Introduction

Week one of a programming course is often lost to installing software, not to programming. Every year, several hundred UNLV students in the introductory C++ course (CS 135) and the x86-64 assembly course (CS 218) spend their first days making a compiler or assembler, a linker, a debugger, and an editor work on dozens of different laptops. Owners of the newest Macs sometimes cannot run the course's 19 GB virtual machine at all and are sent to a campus lab instead. Other universities report the same: 75–100 minutes of class time per semester lost to installs [6], and most help traffic caused by environment mismatch rather than course content [3].

Purpose/Aim

Before the fellowship I built a fifth option, after login servers, per-course virtual machines, cloud editors, and monitored platforms: a Docker container holding each course's exact tools plus the VS Code editor, running entirely on the student's own laptop in a browser tab. One download, one command, no server, no account, nothing recorded. A promise tested only where it succeeds is an advertisement, not a result, so this project asks:

Q1 Does the same package behave identically on Windows, macOS, and Linux, on Intel and ARM chips, on real CS 218 assignments?

Q2 What does it cost the student: download size, start-up time, memory, build time?

Q3 Where does it break, and can we say exactly why?

Methods

What we built. Two containers, one per course, built from Ubuntu Linux with the browser-served VS Code editor and the exact tool versions the instructor chose (g++ for CS 135; yasm, nasm, ld, and gdb for CS 218). Coursework stays in a normal folder on the laptop, never inside the container, so the container can be deleted or updated at any time:

```
docker run -p 127.0.0.1:8218:8080 -v ~/UNLV/x86-workspace:/home/coder/workspace
seancnc/unlv-x86-ide
```

How we tested it. One unchanged script ran on four machines, each playing a fresh student laptop: Windows 11 and Linux PCs on Intel chips, Linux on an ARM chip (all three rented, under $10 in total), and an owned M1 Pro MacBook Pro. It records the image fingerprint, times editor start-up, assembles, links, runs, and debugs a program, builds four real CS 218 assignments (one multithreaded), then deletes the container and checks that the student's files survive.

Results

4 of 4
machines ran all four CS 218 assignments correctly, including the multithreaded one, on the identical image

0.5 s
until the editor opens on Intel chips; about 5 s on Apple Silicon. A course virtual machine takes minutes.

552 MB
one-time download, the identical file on every machine, versus a 19 GB virtual machine

Table 1. What the script recorded on each machine (memory in MiB).

Measure	Windows 11 (Intel)	Linux PC (Intel)	Apple Silicon Mac	Linux on ARM
All four assignments correct	yes	yes	yes	yes
Files survive deleting the container	yes	yes	yes	yes
Debugger	works	works	**silent**	**silent**
Editor ready in (seconds)	0.6	0.5	4.8	6.7
Memory while idle	56	54	263	255
Peak memory during coursework	90	n/a	358	334
Heaviest assignment builds in (s)	0.6	0.3	3.1	3.6
Checks passed / failed	13 / 0	13 / 0	12 / 0	12 / 0

ARM totals exclude the debugger check: under translation the script records the debugger result as a finding rather than a pass or fail, so 12 checks are counted instead of 13.

Figures

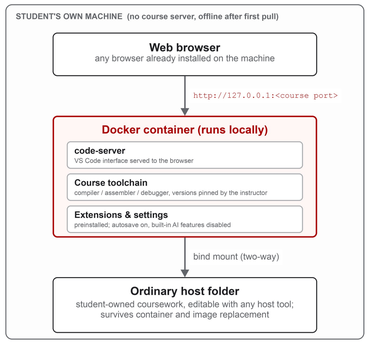
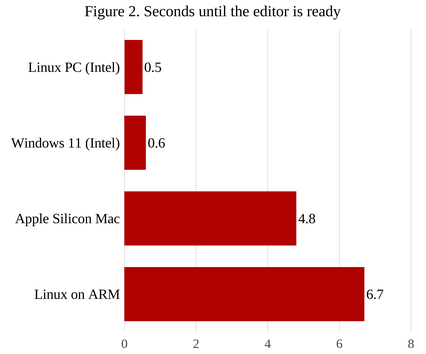


Figure 1. Everything runs on the student's machine; the internet is needed only for the first download.



Intel machines open the editor in about half a second; ARM machines run the Intel-built image through a translator and take 5–7 s.

Discussion

What the numbers mean. On Intel machines the cost is close to zero: the editor is ready in about half a second, idles at roughly 55 MiB, and the heaviest assignment builds in under a second. On ARM chips every assignment still produced the correct answer; start-up rose to 5–7 s and compile-heavy steps ran roughly ten times slower, yet every build finished in under four seconds. The identical 552 MB image was downloaded everywhere, so any difference is due to the machine, not the package.

The one limit. CS 218 teaches the Intel x86-64 instruction set, so that package is built for Intel chips on purpose and ARM chips run it through a translator. The translator cannot support the debugger: on a Mac it appears to run but prints no register values, so the handout tells Apple Silicon owners to finish debugger assignments on an Intel machine or in the lab. It is a translator boundary, not an operating-system boundary.

Conclusions/Future Research

For students and the course. Week one goes to learning, not installing. One 552 MB download replaces a 19 GB virtual machine, works offline afterwards, records nothing, and keeps coursework in a folder the student owns. The department maintains no servers; the instructor updates one recipe and every student gets the same tools. The trade-off is now explicit: keep the real x86-64 instruction set and lose the debugger on ARM.

What comes next. An IRB-approved classroom study in CS 135 and CS 218 measuring how many students reach a working setup without help and how much staff time it saves, using instruments adapted from [3, 4, 6]. Then a hybrid ARM container that runs the editor natively and translates only student programs, which could bring the debugger back to Apple Silicon.

References/Acknowledgements

[1] D. J. Malan. Containerizing CS50. ITiCSE 2024. [3] S. Valstar, W. G. Griswold, L. Porter. Using DevContainers to Standardize Student Development Environments. ITiCSE 2020. [4] K. Fernalld et al. Lightweight Symphony: Reducing CS Student Anxiety with Standardized Docker Environments. SIGITE 2023. [6] D. P. Harvie et al. Using Virtual Machines to Enhance the Educational Experience. SIGITE 2019. [8] H. Park et al. CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring. Applied Sciences 15(19), 2025. [12] UNLV Dept. of Computer Science. Student Center: Remote Access and File Storage. tux.cs.unlv.edu, accessed Aug. 2026.

Supported by the UNLV Office of Undergraduate Research Summer Undergraduate Research Fellowship. Every number on this poster comes from the published test script's output files, archived in the repository under tests/record-results.